

Universidade de Trás-os-Montes e Alto Douro

Ricardo Rodrigues Pinheiro

AL77167



Vila Real, 2024

Universidade de Trás-os-Montes e Alto Douro

Engenharia Informática

Relatório de Projeto de Licenciatura em Engenharia Informática:

Sistema de Detecção de Peões para a Prevenção de Atropelamento

Ricardo Rodrigues Pinheiro AL77167

Orientador:

Vítor Manuel de Jesus Filipe

Coorientador:

Cristiano Gonçalves Pendão



Vila Real, 2024

Resumo

Em 2023, registaram-se 8.936 acidentes envolvendo motociclos em Portugal, um número que tem vindo a aumentar anualmente. Este projeto visa reduzir o número de acidentes rodoviários com motociclos e mitigar o risco de colisão com peões através da utilização de inteligência artificial, que tem demonstrado grande potencial nesta área. O projeto baseia-se na análise de imagem capturada por uma única câmara, dispensando o uso de sensores adicionais. Esta abordagem é inspirada no sucesso da Tesla, que confia exclusivamente na visão para a navegação e segurança dos seus veículos.

A metodologia aplicada neste projeto inclui a utilização de técnicas avançadas de processamento de imagem para análise de vídeo em tempo real, permitindo alertar rapidamente o condutor sobre possíveis perigos. O relatório apresenta uma visão detalhada do problema abordado, descreve a metodologia implementada e discute os resultados obtidos. As conclusões indicam que o uso exclusivo de imagem é eficaz e viável, contribuindo significativamente para a segurança rodoviária e a redução de acidentes com motociclos.

Índice

CAPÍTULO I – Introdução	6
1.1 Objetivo	6
1.2 Planeamento	7
CAPÍTULO II – Enquadramento Teórico	8
2.1 O que é uma CNN	8
2.2 YOLO	9
CAPÍTULO III – Trabalho Desenvolvido	11
3.1 Datasets	11
3.2 Treino do modelo	12
3.3 Teste do modelo	13
3.4 Algoritmo	17
CAPÍTULO IV – Conclusão	21
CAPÍTULO V- Referências	22

Índice de figuras

Fig. 1-Arquitetura do YOLOv8.....	10
Fig. 2-Comparação da mAP a 0.5:0.95	13
Fig. 3-Comparação da mAP a 0.5	14
Fig. 4-Comparação da Recall	15
Fig. 5-Comparação da Precisão	15
Fig. 6-Matriz de confusão do modelo treinado	16
Fig. 7-Demonstração do algoritmo	19

CAPÍTULO I – Introdução

Neste capítulo procede-se à Introdução ao trabalho que permitiu descrever o
Objetivo (1.1) e o Planeamento (1.2)

1.1 Objetivo

O objetivo deste trabalho é desenvolver um modelo de Inteligência Artificial (IA) capaz de detetar peões e um algoritmo que consiga prever o risco de colisão. A criação deste modelo visa aumentar a segurança em ambientes urbanos, onde a interação entre veículos e pedestres é intensa e constante. Ao usar câmaras na mota elétrica desenvolvida no projeto A-MoVeR, espera-se que o sistema de IA possa identificar a presença de peões em tempo real, analisando dados visuais para reconhecer padrões e comportamentos que indiquem a presença de pessoas.

Além da deteção de peões, o projeto procura desenvolver um algoritmo avançado que consiga prever a colisão. Este algoritmo analisará o ambiente à frente do veículo e o posicionamento dos peões, avaliando a probabilidade de acidentes e emitindo alertas preventivos para o condutor. O uso desta tecnologia permitirá uma resposta proativa e rápida, potencialmente evitando acidentes antes que eles aconteçam.

Ao atingir estes objetivos, o trabalho contribuirá significativamente para a segurança no trânsito urbano, proporcionando uma solução tecnológica que pode ser aplicada em veículos de entrega e outras frotas urbanas, promovendo um ambiente mais seguro e eficiente para todos.

1.2 Planeamento

O planeamento deste projeto consiste numa abordagem sistemática e abrangente, pretendendo a criação de um sistema de apoio à condução para motociclos, focado na deteção de peões e na prevenção de atropelamentos. Inicialmente, foi realizado um levantamento exaustivo dos sistemas de apoio à condução existentes ou aplicáveis a motociclos, com o objetivo de identificar as melhores práticas e tecnologias disponíveis.

Uma parte fundamental do projeto consistiu no estudo e seleção de algoritmos e modelos de inteligência artificial específicos para a deteção de peões em ambientes rodoviários. Foram estudados alguns modelos e técnicas de *machine learning*, com o intuito de escolher os modelos mais adequados para o sistema em questão. Além disso, foi realizado um esforço significativo na identificação e construção de *datasets* de alta qualidade para o treino e validação do algoritmo selecionado.

Com base nos algoritmos e *datasets* escolhidos, foi elaborada uma solução de apoio à condução que integre a deteção de peões e o alerta para o perigo de atropelamento. Esta solução foi projetada de forma a ser eficiente, precisa e intuitiva para o utilizador, garantindo uma experiência de condução mais segura e confiável. Por fim, foi realizada a implementação e validação do algoritmo/modelo selecionado, onde foram realizados testes rigorosos em condições reais de condução para avaliar o desempenho e a eficácia do sistema desenvolvido. Este processo garante que o sistema atenda aos padrões de segurança e qualidade exigidos, contribuindo assim para a redução de acidentes rodoviários envolvendo motociclos.

CAPÍTULO II – Enquadramento Teórico

2.1 O que é uma CNN

Uma *Convolutional Neural Network* (CNN), ou Rede Neuronal Convolucional, é um tipo de rede neuronal artificial especialmente projetada para o processamento de imagens. Ela é composta por várias camadas de neurônios, organizadas em três tipos principais: camadas de convolução, camadas de *pooling* e camadas totalmente conectadas.

Na CNN, as camadas de convolução são responsáveis por aplicar filtros sobre a entrada da imagem. Cada filtro é uma pequena matriz que desliza sobre a imagem, realizando operações de convolução para extrair características importantes, como bordas, texturas e padrões. Essas características são então ativadas por funções não-lineares, como a função ReLU (Rectified Linear Unit), para introduzir não-linearidade na rede.

Após as camadas de convolução, as camadas de pooling são aplicadas para reduzir a dimensionalidade da imagem e extrair as características mais importantes de forma mais eficiente. Isso é feito através de operações como o max pooling, que seleciona o valor máximo de uma região da imagem, preservando as características mais proeminentes.

Finalmente, as camadas totalmente conectadas agrupam todas as características extraídas pela rede e utilizam para realizar a classificação ou detecção desejada. Estas camadas são semelhantes às camadas de neurônios em redes neuronais tradicionais, onde cada neurônio está conectado a todos os neurônios da camada anterior.

Em resumo, uma CNN utiliza a convolução para extrair características importantes das imagens, reduz a dimensionalidade através de operações de pooling e utiliza camadas totalmente conectadas para realizar a classificação ou detecção final. Esta arquitetura é altamente eficaz para a detecção de padrões em imagens e tem sido amplamente utilizada em aplicações como reconhecimento de objetos, detecção de peões e condução autónoma.

2.2 YOLO

A primeira versão, YOLOv1, introduziu o conceito de dividir a imagem em várias partes e fazer previsões de *bounding boxes* e classes de objetos em cada parte. Este modelo já mostrava um bom equilíbrio entre velocidade e precisão. Nas versões subsequentes, YOLOv2 e YOLOv3, foram implementadas melhorias significativas, como o uso de redes maiores e aprimoradas, além de aumentos na resolução das imagens. Com o lançamento do YOLOv8, o modelo continuou a evoluir, integrando tecnologias de ponta e abordagens inovadoras para aumentar a eficiência e a precisão da detecção de objetos.

Funcionamento do YOLOv8

Divisão da Imagem em Partes: A imagem de entrada é dividida em uma grade de retângulos. Cada retângulo, ou célula, é responsável por prever a presença de objetos naquela região específica. Isso permite que o modelo analise diferentes áreas da imagem simultaneamente, aumentando a eficiência do processo de detecção.

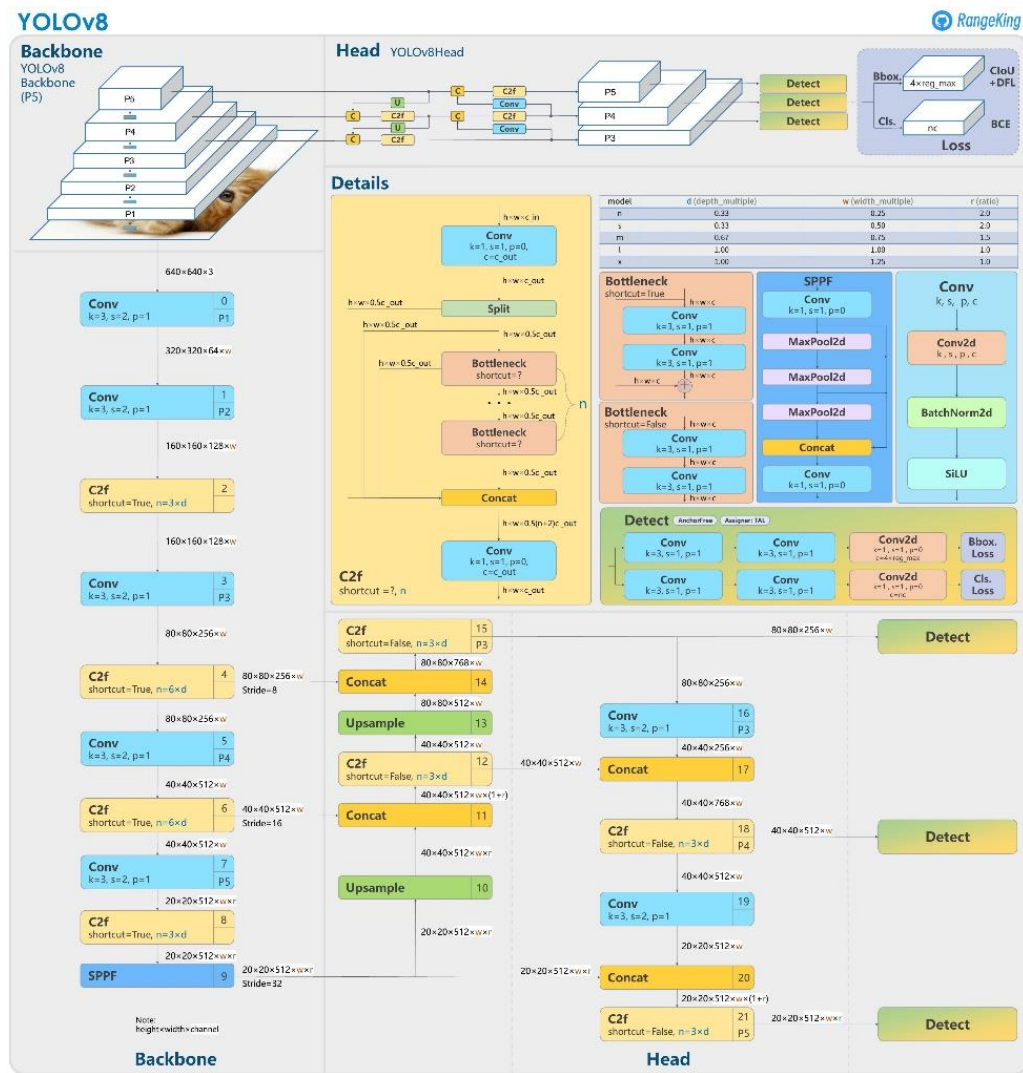
Extração de Recursos: Redes convolucionais (CNNs) são utilizadas para extrair características visuais de cada célula. Essas características representam os padrões, texturas e formas presentes na imagem. A extração de recursos é crucial, pois fornece as informações necessárias para identificar e localizar objetos.

Previsão de *Bounding Boxes* e Classes: Para cada célula da grade, o YOLOv8 prevê uma caixa delimitadora (*bounding box*) e a classe do objeto contido nela. A *bounding box* indica a localização e o tamanho do objeto, enquanto a classe identifica o tipo de objeto (por exemplo, carro, pessoa, bicicleta). Esta abordagem permite ao modelo fazer previsões rápidas e precisas para cada região da imagem.

Deteção de Objetos: O algoritmo combina as previsões de *bounding box* e classe para identificar objetos na imagem. A cada detecção, é atribuída uma confiança, representando a probabilidade de um objeto estar presente naquela localização específica. Este processo garante que o modelo possa diferenciar entre objetos de diferentes classes e localizar cada um com precisão.

Supressão de Não Máximo (NMS): Para eliminar *bounding boxes* sobrepostas que representam o mesmo objeto, o NMS é aplicado. Este processo seleciona a *bounding box* com maior confiança e remove as outras que se sobrepõem a ela acima de um limiar definido. A aplicação do NMS garante uma detecção mais precisa e sem redundâncias, refinando ainda mais os resultados do modelo.

Com estas etapas, o YOLOv8 consegue realizar detecções de objetos de forma eficiente e precisa, sendo a ferramenta perfeita para as necessidades deste projeto.



CAPÍTULO III – Trabalho Desenvolvido

Neste capítulo procede-se à explicação do trabalho desenvolvido: Datasets (3.1), Treino do Modelo(3.2), Teste do modelo (3.3) e Algoritmo (3.4)

3.1 Datasets

O primeiro passo para desenvolver este modelo foi procurar ou criar um bom *dataset* para treinar o modelo de detecção de objetos. A criação de um *dataset* envolve gravar vídeos ou tirar fotos durante a condução e, posteriormente, anotar (marcar a posição) das pessoas nas imagens. No entanto, como é necessário um grande volume de imagens e o processo de anotação é bastante demorado, o ideal seria utilizar um *dataset* já existente e bem estabelecido.

Durante a minha pesquisa, encontrei o *dataset* da [nuScenes](#), um recurso extenso e bem documentado que contém uma grande quantidade de imagens anotadas para detecção de objetos. O *dataset* da [nuScenes](#), gravado em Boston e Singapura, possui um total de 63.859 imagens, das quais apenas 34.831 contêm pedestres. Contudo, o *dataset* da [nuScenes](#) não estava no formato esperado pelo YOLOv8. O formato esperado pelo YOLOv8 inclui arquivos de texto com as anotações das *bounding boxes*, onde cada linha representa um objeto com as informações da classe a que pertence, coordenadas da caixa delimitadora e dimensões normalizadas em relação à imagem.

Para adaptar o *dataset* da [nuScenes](#) ao formato do YOLO, foi necessário realizar manipulações nas anotações originais. Isso envolveu a conversão dos dados de anotação fornecidos pela [nuScenes](#) para o formato de texto específico utilizado pelo YOLO. Cada anotação precisou ser transformada para especificar a classe do objeto, bem como as coordenadas da *bounding box* (x, y, largura e altura), todas normalizadas em relação ao tamanho da imagem.

Esta conversão foi um passo crucial, pois garantiu que o *dataset* estivesse corretamente formatado que era essencial para treinar o modelo de maneira eficaz. Utilizando scripts de conversão, consegui transformar as anotações do *dataset* da nuScenes no formato necessário. Com o *dataset* agora pronto para uso, o próximo passo foi iniciar o processo de treinamento do modelo YOLOv8, aproveitando a vasta quantidade de dados já anotados para obter uma detecção de objetos precisa e confiável.

Ao utilizar um *dataset* bem estabelecido como o nuScenes, pude economizar tempo valioso e concentrar esforços na implementação e ajuste do modelo, garantindo que ele estivesse bem preparado para a tarefa de detecção de pedestres em cenários de condução.

3.2 Treino do modelo

Nesta fase, iniciei o processo de treino do modelo. Para isso, foi necessário primeiro realizar a limpeza e preparação do *dataset*. Esta etapa envolveu a remoção de imagens e anotações inconsistentes ou irrelevantes. Além disso, as imagens e anotações precisaram ser divididas em conjuntos de treino (80%) e validação (20%). Esta divisão é fundamental para avaliar o desempenho do modelo de maneira objetiva e evitar o *overfitting*, que ocorre quando o modelo se ajusta excessivamente aos dados de treino e perde a capacidade de generalizar para novos dados. Além dos conjuntos de treino e validação, também utilizei um conjunto de teste, composto por vídeos gravados e anotados por mim, para uma avaliação final do modelo.

Para o treino inicial do modelo, utilizei o meu próprio computador. No entanto, devido ao elevado número de imagens e à alta qualidade dessas imagens, o tempo necessário para treinar cada época (*epoch*) era excessivamente longo. Uma época é uma iteração completa sobre todo o *dataset* de treino, e treinar com muitas épocas é essencial para que o modelo possa aprender de maneira eficaz. Devido às limitações de hardware do meu computador, o treino era impraticável.

Como solução, passei a utilizar o Google Colab, que oferece acesso a GPUs potentes durante tempo limitado. Inicialmente, realizei alguns testes para determinar os melhores parâmetros de treino. Esses testes foram cruciais para otimizar o desempenho do modelo sem desperdiçar recursos computacionais.

Decidi treinar o modelo com a resolução total das imagens para preservar a riqueza dos detalhes visuais, mesmo que isso significasse trabalhar com um número reduzido de parâmetros. Optar por menos parâmetros envolveu ajustes específicos, como a redução da complexidade do modelo e a limitação do número de camadas e neurônios, para garantir que o treino fosse eficiente e concluído em um tempo razoável.

Essa abordagem equilibrada entre a qualidade das imagens e a complexidade do modelo permitiu um treino eficaz, resultando em um modelo robusto capaz de detetar pedestres com alta precisão. O uso do Google Colab foi fundamental para superar as limitações de hardware e acelerar significativamente o processo de treino, possibilitando ajustes e melhorias contínuas até alcançar um desempenho satisfatório.

Ao todo, o treino do modelo consistiu em 25 épocas, cada uma demorou aproximadamente 1 hora para ser concluída. No entanto, devido à restrição de tempo do Google Colab, que permite apenas 3 horas de uso semanal de computação gratuita, o processo tornou-se demorado e desafiador. Gerenciar o tempo de computação disponível e otimizar os parâmetros de treinamento dentro dessas limitações foi uma tarefa complexa.

Essa restrição de tempo adicionou uma camada extra de complexidade ao processo de treino, exigindo um planeamento cuidadoso e uma abordagem estratégica para aproveitar ao máximo o tempo disponível.

Apesar dos desafios enfrentados devido às limitações de tempo, o uso do Google Colab permitiu superar as barreiras de hardware e realizar um treino eficaz do modelo.

3.3 Teste do modelo

O YOLOv8 já vem com modelos pré-treinados, o que proporciona uma base sólida para iniciar o treino e ajustar o modelo às necessidades específicas do projeto. Para identificar o melhor modelo pré-treinado a ser utilizado e garantir as melhores detecções, realizei um teste utilizando um vídeo gravado e anotado por mim. Em seguida, testei o modelo que treinei e um pré-treinado, utilizando uma RTX 3060 laptop, e apresentei os resultados obtidos com uma análise detalhada de cada métrica avaliada.

No teste do modelo treinado, os resultados foram os seguintes: o tempo de inferência, que é o tempo necessário para o modelo processar cada imagem e fazer as previsões, foi de 16,8ms por imagem, enquanto o tempo de pós-processamento, que envolve as etapas adicionais após a inferência, como o desenho das *bounding boxes*, foi de 5,2ms por imagem.

No teste do YOLOv8n pré-treinado, os resultados foram os seguintes: o tempo de inferência foi de 15,5ms por imagem e o tempo de pós-processamento foi de 6,4ms por imagem.

Análise dos Resultados

Os gráficos fornecem uma visão clara das métricas de desempenho dos diferentes modelos pré-treinados. As principais métricas avaliadas foram:

1. mAP at IoU=0.50:0.95:

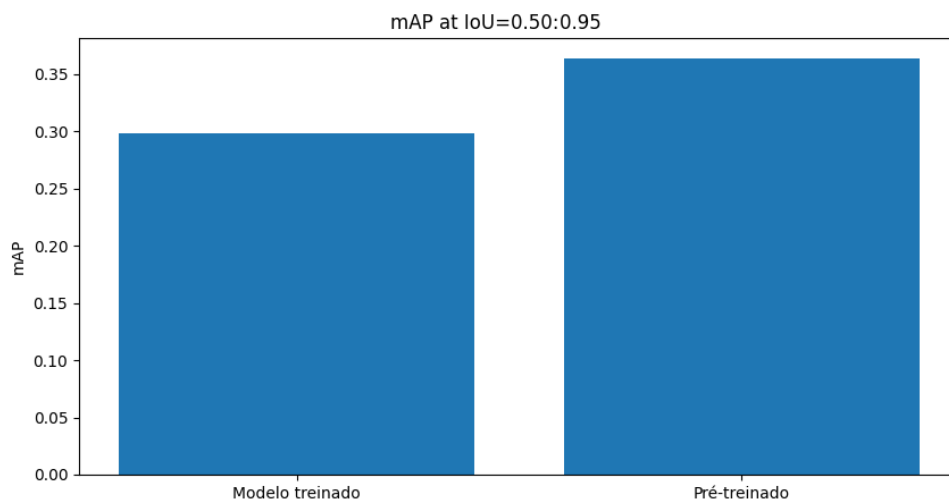


Fig. 2-Comparação da mAP a 0.5:0.95

- Definição: *Mean Average Precision* (mAP) é a média das precisões calculadas em vários limiares de *Intersection over Union* (IoU). O IoU é uma métrica que avalia a sobreposição entre a caixa delimitadora (*bounding box*) prevista e a caixa delimitadora real do objeto.
- Cálculo: Para cada limiar de IoU (0.50, 0.55, 0.60, ..., 0.95), a precisão média é calculada e, em seguida, a média dessas precisões é tomada.
- Interpretação: Valores de mAP mais altos indicam melhor desempenho do modelo, pois refletem maior precisão na detecção de objetos em diversos limiares de IoU.

2. mAP at IoU=0.50:

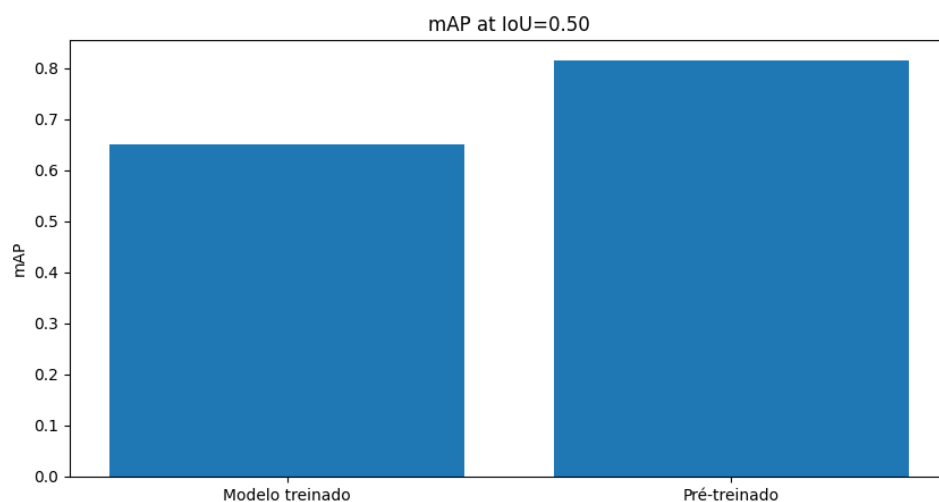


Fig. 3-Comparação da mAP a 0.5

- Definição: Esta métrica é a precisão média para um IoU fixo de 0.50.
- Cálculo: A precisão é calculada considerando apenas o limiar de IoU de 0.50.
- Interpretação: Um valor de mAP mais alto nesta métrica indica que o modelo é capaz de detetar objetos com precisão quando a sobreposição entre as caixas preditas e as reais é de pelo menos 50%.

3. Recall:

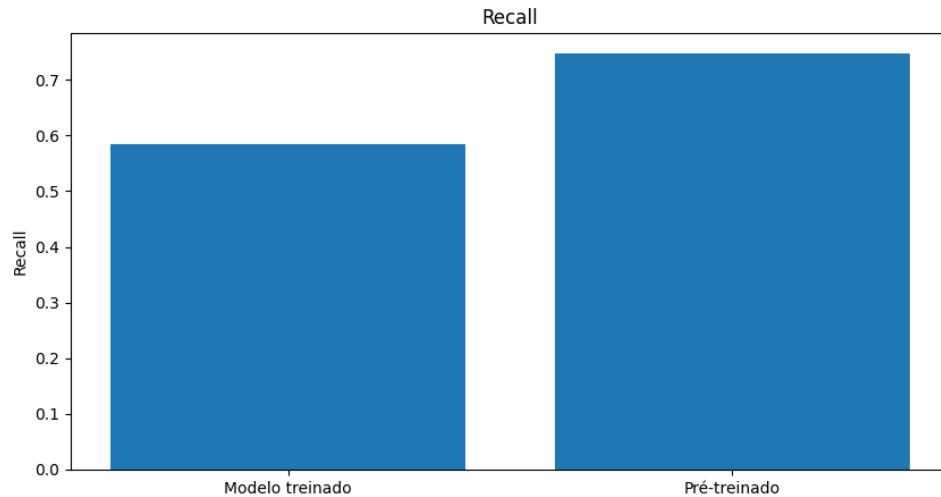


Fig. 4-Comparação da Recall

- Definição: O *Recall* (ou sensibilidade) mede a capacidade do modelo de encontrar todas as instâncias relevantes em um *dataset*.
- Cálculo: *Recall* é a razão entre o número de verdadeiros positivos e a soma dos verdadeiros positivos e falsos negativos.
- Interpretação: Um valor de *recall* mais alto indica que o modelo está identificando a maioria dos objetos presentes nas imagens, minimizando as omissões.

4. Precision:

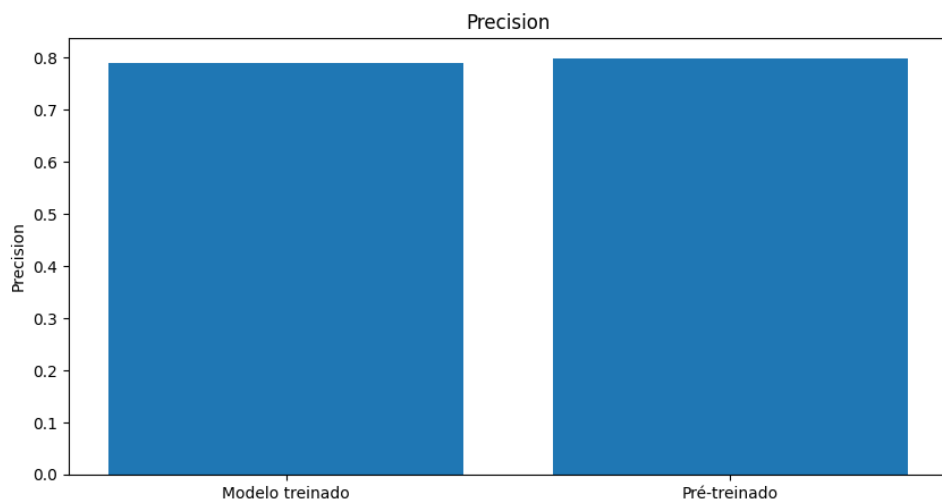


Fig. 5-Comparação da Precisão

- Definição: *Precision* mede a proporção de verdadeiros positivos em relação ao total de previsões positivas feitas pelo modelo.

- Cálculo: *Precision* é a razão entre o número de verdadeiros positivos e a soma dos verdadeiros positivos e falsos positivos.
- Interpretação: Um valor de *precision* mais alto indica que o modelo tem uma alta taxa de acertos em suas previsões, com poucos falsos positivos.

Matriz de Confusão

A matriz de confusão do modelo treinado apresentada a seguir fornece uma visualização do desempenho do modelo de detecção de pedestres em termos de classificações corretas e incorretas, tanto para a classe "pedestre" quanto para "background".

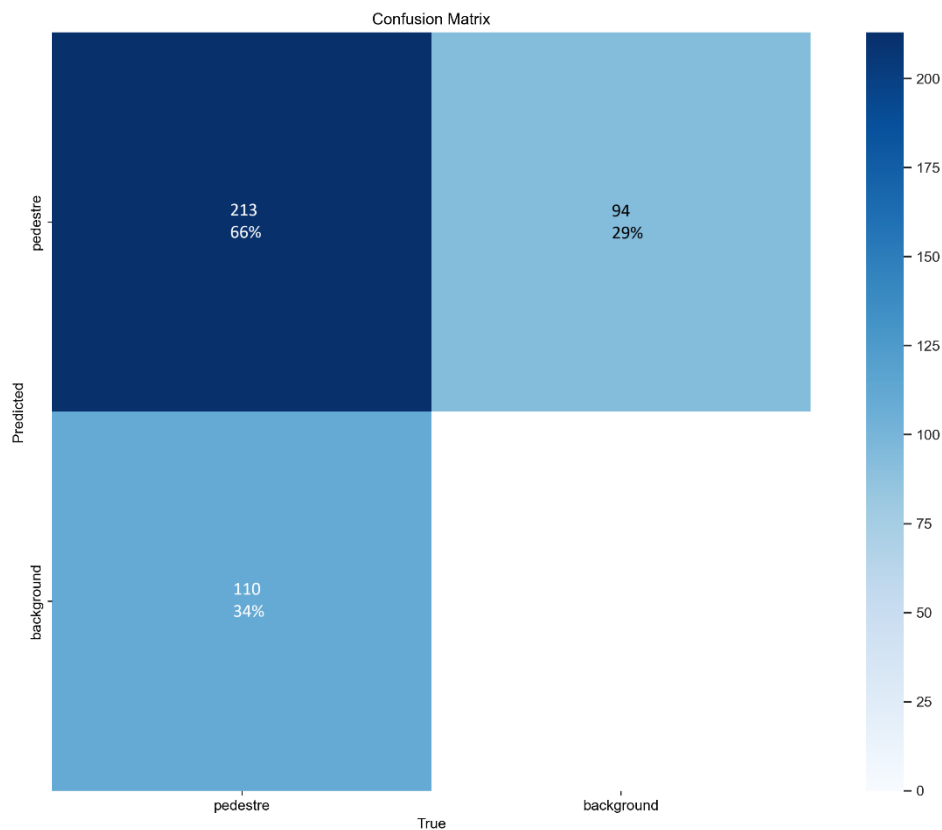


Fig. 6-Matriz de confusão do modelo treinado

True Positives (TP): 213 pedestres corretamente identificados como pedestres (66%).

False Negatives (FN): 110 pedestres incorretamente identificados como background (34%).

False Positive (FP): 94 pedestres inexistentes incorretamente identificados como pedestres (29%).

Na matriz as percentagens estão em relação ao número de *bounding boxes* existentes, pedestres anotados (323).

Esta matriz de confusão indica que o modelo tem uma precisão razoável na detecção de pedestres

Resultados e Interpretação

As figuras ilustram os valores de mAP at IoU=0.50:0.95, mAP at IoU=0.50, *Recall* e *Precision* para os diferentes modelos pré-treinados. Observando os resultados:

- **mAP at IoU=0.50:0.95:** O modelo pré-treinado apresentou valores mais altos (0.298 vs. 0.364), indicando uma melhor precisão média em diferentes limiares de IoU.
- **mAP at IoU=0.50:** Novamente, o modelo pré-treinado destacou-se (0.651 vs. 0.814), mostrando que é eficiente na detecção de objetos com um IoU de pelo menos 50%.
- **Recall:** O modelo pré-treinado também apresentou um *recall* superior (0.585 vs. 0.747), demonstrando sua capacidade de identificar a maioria dos objetos presentes nas imagens.
- **Precision:** A precisão do modelo pré-treinado (0.791 vs. 0.798) foi ligeiramente melhor, sugerindo que o modelo é eficaz em minimizar os falsos positivos.

Com base nos testes realizados e nas figuras analisadas, o modelo YOLOv8 pré-treinado mostrou-se superior em todas as métricas avaliadas. Esse desempenho pode ser atribuído ao fato de que os modelos pré-treinados do YOLOv8 foram desenvolvidos e ajustados com um grande volume de dados e recursos computacionais avançados. Por outro lado, o modelo treinado especificamente para este projeto teve um número limitado de épocas devido a restrições de tempo e recursos, o que pode ter contribuído para o seu desempenho inferior.

A superioridade do modelo YOLOv8 pré-treinado em termos de mAP, *recall* e precisão destaca a sua eficácia na detecção de objetos em ambientes de condução por isso optei por usar este modelo para o desenvolvimento do algoritmo.

3.4 Algoritmo

Para a implementação do algoritmo de detecção de pedestres, inicialmente pesquisei diversas abordagens e técnicas que poderiam ser utilizadas. Uma das metodologias estudadas e testadas foi a técnica de rastreamento de pedestres (*pedestrian tracking*). A seguir, explico como funciona essa técnica e por que decidi não a utilizar.

O rastreamento de pedestres é uma técnica que segue a posição dos pedestres ao longo do tempo em uma sequência de imagens ou vídeos. Esta técnica geralmente envolve os seguintes passos:

1. **Deteção Inicial:** Primeiro, os pedestres são detetados em um *frame* específico utilizando o YOLO.
2. **Associação de Objetos:** Após a deteção inicial, cada pedestre detetado é associado a uma identidade única. Basicamente é lhe dado um Id.

3. **Rastreamento:** Nos *frames* subsequentes, o algoritmo volta a localizar o pedestre e coloca um ponto na sua nova posição. Isto faz com que seja criado uma linha correspondente ao posicionamento do pedestre.

No entanto, ao aplicar essa técnica em um cenário de condução, onde o veículo está em movimento, surgiram alguns problemas específicos:

- **Movimento da Câmara:** Como a câmara está fixada no veículo e este está em constante movimento, o cenário visual muda continuamente. Isso pode fazer com que objetos fixos, como pessoas sentadas em um ponto específico, aparentem estar em movimento relativo devido à mudança de perspectiva da câmara. Este fenômeno dificulta a distinção entre pedestres que realmente estão em movimento e objetos estáticos no ambiente.
- **Oclusão e Desaparecimento Temporário:** Em um ambiente dinâmico, pedestres podem ser momentaneamente ocultados por outros objetos ou sair do campo de visão da câmara, complicando o rastreamento contínuo e preciso.

Devido a estes desafios, optei por abandonar a implementação baseada em rastreamento contínuo e focar em uma abordagem de detecção por regiões de interesse, que se mostrou mais robusta e adequada para o contexto do projeto. Esta abordagem, embora mais simples, permite uma detecção precisa e confiável de pedestres em cada *frame*.

Funcionamento do Algoritmo criado

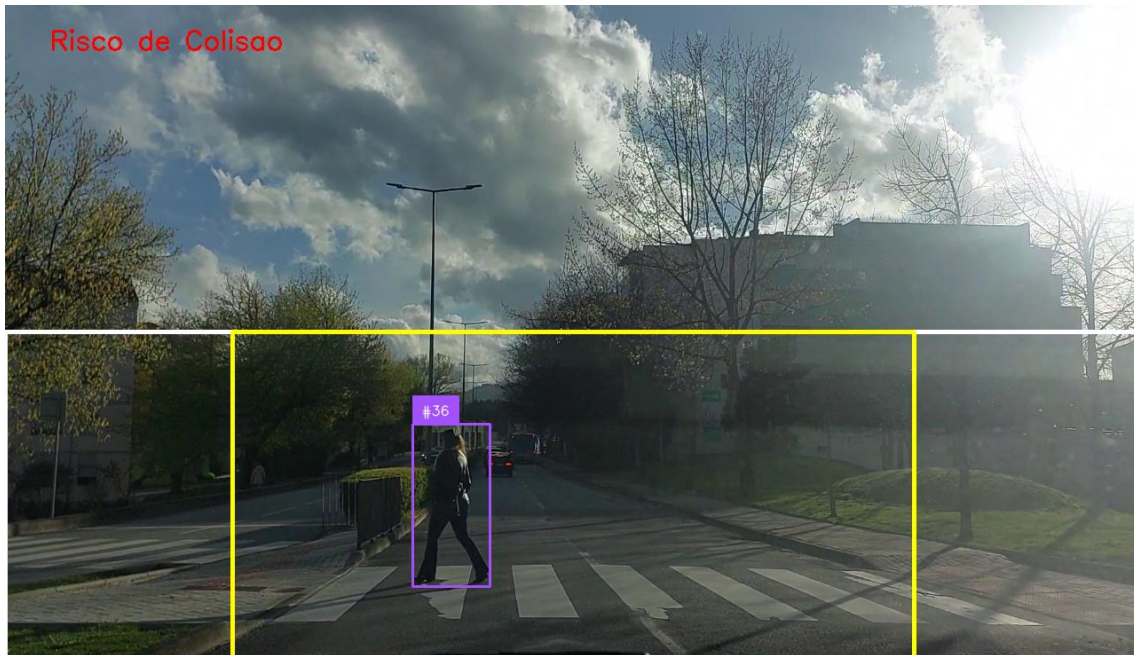


Fig. 7-Demonstração do algoritmo

O algoritmo desenvolvido para detecção de pedestres funciona da seguinte maneira, conforme ilustrado na imagem anexa:

- **Deteção de Pedestres:**
 - O primeiro passo é a detecção dos pedestres utilizando o modelo pré-treinado YOLOv8. Este modelo identifica e anota (localiza) cada pedestre na imagem, criando uma *bounding box* ao redor de cada indivíduo.
- **Definição de Regiões de Interesse (ROI):**
 - Após a detecção, são definidas regiões de interesse (ROI) na imagem. Essas regiões são áreas específicas onde a presença de pedestres é monitorizada.
- **Verificação nas Regiões de Interesse:**
 - O algoritmo verifica se o pedestre detetado está dentro de alguma das regiões de interesse. Se um pedestre estiver dentro de uma ROI e a área de sua *bounding box* for maior que um determinado limiar (*threshold*) específico para essa região, um alerta é emitido.
 - **ROI Principal:** Localizada no centro da imagem, indicando maior risco de colisão. Se um pedestre é detetado nesta região, é emitido um alerta de risco de colisão.

- **Outras ROIs:** Se um pedestre é detetado em outras regiões de interesse, é emitido um alerta para que o condutor fique atento.
- **Monitorização do Crescimento da *Bounding Box*:**
 - Além de emitir alertas com base na presença e tamanho da *bounding box*, o algoritmo também monitoriza o crescimento da área ocupada pela *bounding box* ao longo dos *frames*.
 - Cada *bounding box* é identificada com um ID único e a sua área é guardada. Se o crescimento da área da *bounding box* ultrapassar um determinado limiar e a *bounding box* estiver dentro de uma ROI, um alerta de atenção é emitido.
- **Processamento de Vídeo em Tempo Real:**
 - O algoritmo funciona com vídeo em tempo real, garantindo respostas rápidas e precisas. Para otimizar o uso de recursos computacionais, o vídeo é processado apenas de 3 em 3 *frames*. Esta abordagem reduz a carga de processamento sem comprometer a eficácia da deteção, uma vez que não é necessário analisar todos os *frames* para manter uma monitorização eficiente.
- **Emissão de Alertas:**
 - Quando um pedestre é detetado dentro de uma ROI e os critérios de área e crescimento da *bounding box* são atendidos, o sistema emite alertas de forma imediata. Estes alertas podem ser visuais, destinados a chamar a atenção do condutor para a presença de pedestres e potencial risco de colisão.

Este método garante que o sistema não só detete a presença de pedestres, mas também monitorize a aproximação dos mesmos, emitindo alertas que aumentam a segurança do condutor e dos pedestres. Ao processar vídeo em tempo real e analisar apenas *frames* selecionados, o algoritmo otimiza o uso de recursos computacionais enquanto mantém uma alta precisão na deteção de pedestres e na prevenção de colisões.

Este sistema integrado de deteção e monitorização contínuo é crucial para a segurança no trânsito, especialmente em ambientes urbanos movimentados onde a presença de pedestres é constante e imprevisível.

CAPÍTULO IV – Conclusão

Através do desenvolvimento e implementação deste algoritmo para a detecção de pedestres, conseguimos alcançar resultados promissores na melhoria da segurança rodoviária, especialmente para motociclistas. Utilizando o modelo pré-treinado YOLOv8, conseguimos criar um sistema eficiente que não só detecta pedestres, mas também monitoriza a sua aproximação e emite alertas em tempo real. Este sistema foi projetado para otimizar os recursos computacionais ao processar apenas *frames* selecionados, mantendo, ao mesmo tempo, uma alta precisão e eficiência na detecção.

Os testes realizados demonstraram que esta abordagem é altamente eficaz na prevenção de colisões, com o algoritmo respondendo rapidamente às mudanças no ambiente e emitindo alertas conforme necessário. A definição de regiões de interesse (ROI) e o monitoramento do crescimento das *bounding boxes* ao longo dos *frames* foram elementos cruciais que contribuíram para a robustez do sistema.

Apesar dos bons resultados obtidos, é necessário considerar ajustes futuros para garantir a eficácia do algoritmo em diferentes contextos. A principal modificação recomendada é o ajuste dos valores de limiares (*thresholds*) para se adequar às diferentes câmaras que poderão ser utilizadas quando o sistema for implementado em cenários reais. Este ajuste é essencial para garantir que o sistema mantenha a sua precisão e capacidade de resposta independentemente das variáveis específicas de cada câmara utilizada.

Em resumo, a solução desenvolvida mostrou-se promissora, oferecendo potencial para contribuir com a segurança no trânsito. No entanto, os resultados obtidos até o momento são preliminares e indicam a necessidade de ajustes adicionais. Para que o sistema esteja realmente preparado para aplicações em cenários reais, serão necessários muitos mais testes e ajustes, especialmente no que diz respeito aos valores de limiares (*thresholds*) para diferentes câmaras.

O próximo passo consiste em realizar mais testes e continuar a melhorar a solução, utilizando diversos *datasets* e cenários diferentes. Este processo permitirá ajustar o algoritmo para aumentar a sua precisão e robustez, garantindo que ele seja eficaz em uma ampla gama de situações do mundo real.

CAPÍTULO V- Referências

Terven, J., & Cordova-Esparza, D. (2023). A comprehensive review of YOLO: From YOLOv1 to YOLOv8 and beyond. arXiv preprint arXiv:2304.00501.

(2024). Nuscenes.org.
<https://www.nuscenes.org/nuscenes?externalData=all&mapData=all&modalities=Any>

RangeKing. "Brief Summary of YOLOv8 Model Structure." GitHub, 10 Jan. 2023, github.com/ultralytics/ultralytics/issues/189.

Wang, Z. J., Turko, R., Shaikh, O., Park, H., Das, N., Hohman, F., ... & Chau, D. H. P. (2020). CNN explainer: learning convolutional neural networks with interactive visualization. IEEE Transactions on Visualization and Computer Graphics, 27(2), 1396-1406.

Ultralytics. "YOLOv8." Docs.ultralytics.com, 12 Nov. 2023, docs.ultralytics.com/models/yolov8/.

Diwan, T., Anirudh, G., & Tembhurne, J. V. (2023). Object detection using YOLO: Challenges, architectural successors, datasets and applications. multimedia Tools and Applications, 82(6), 9243-9275.